

Computer Organization & Architecture

Unit- 7

Pipeline And Vector Processing (Part - 2)

Prepared By: Sweetu D. Sureja

Information Technology Department

V.V.P. Engineering College, Rajkot.

- Flynn's taxonomy
- Parallel Processing
- Pipelining
- Arithmetic Pipeline
- **Instruction Pipeline**
- **RISC Pipeline**
- **Vector Processing**
- **Array Processors**

Instruction Pipeline

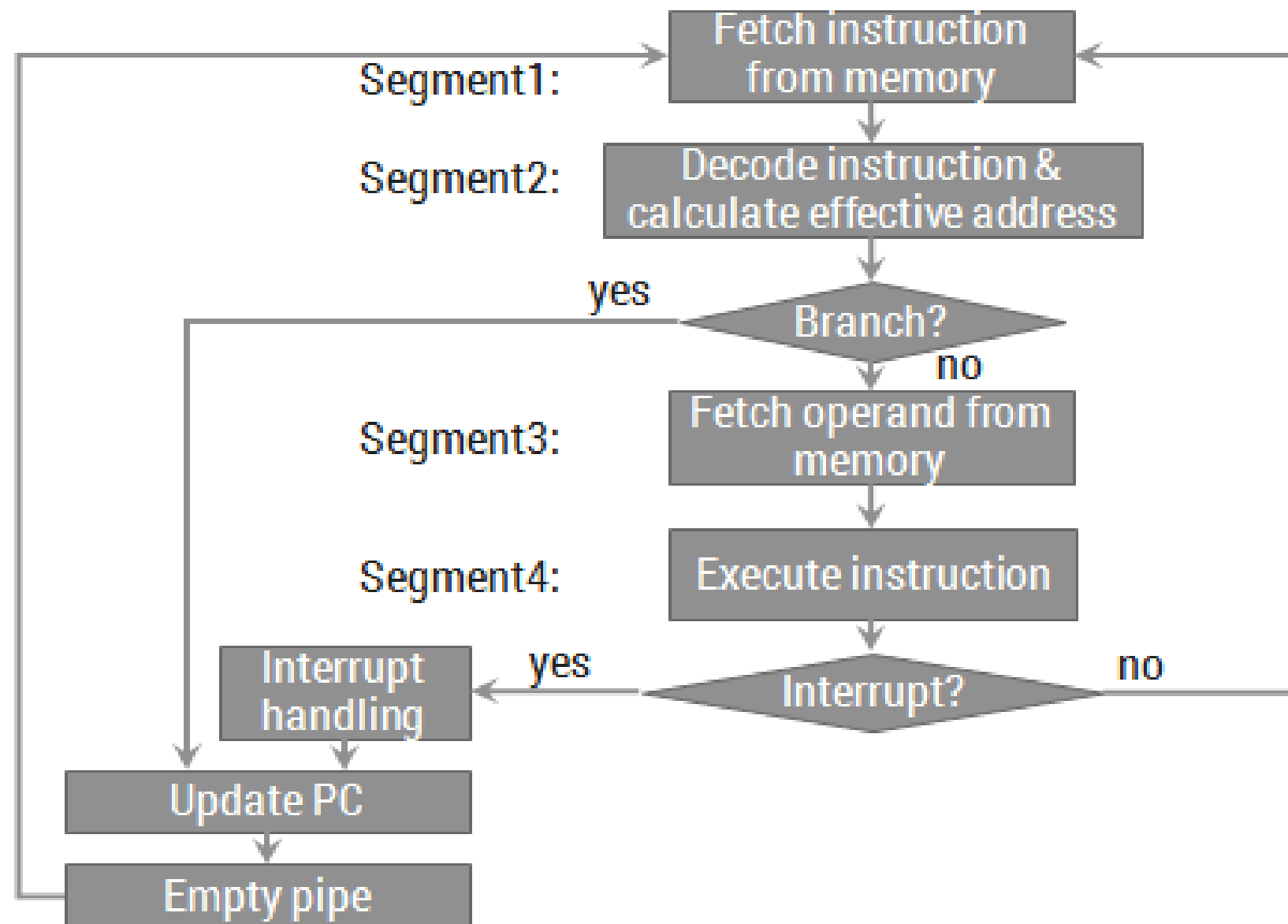
- An instruction pipeline reads consecutive instructions from memory while previous instructions are being executed in other segments.

Example: Four - Segment Instruction Pipeline

- In the most general case, the computer needs to process each instruction with the following sequence of steps.

- FI segment fetches the instruction.
- DA segment decodes the instruction and calculate the effective address.
- FO segment fetches the operand.
- EX segment executes the instruction.

Four segment CPU pipeline



Space-time Diagram

Step:	1	2	3	4	5	6	7	8	9	10	11	12	13
Instruction 1	FI	DA	FO	EX									
2		FI	DA	FO	EX								
(Branch) 3			FI	DA	FO	EX							
4				FI	-	-	FI	DA	FO	EX			
5					-	-	-	FI	DA	FO	EX		
6									FI	DA	FO	EX	
7										FI	DA	FO	EX

- The diagram shows how instructions are executed using a **pipeline**.
- Each instruction goes through 4 steps: **FI (Fetch)**, **DA (Decode)**, **FO (Fetch Operand)**, **EX (Execute)**.
- Multiple instructions run **at the same time** but in different stages → this makes execution faster.
- Instruction 1** starts first and moves step by step.
- Instruction 2, 3, ...** start in the next cycles.
- Instruction 3 is a branch instruction**, so it changes the normal flow.
- Because of this, some steps show “-” (**empty**) → these are **pipeline stalls (waiting time)**.
- After the branch is resolved, execution continues normally with next instructions.

Pipeline increases speed by overlapping instructions, but branch instructions create delays (stalls).

Pipeline Conflicts or Summarize major hazards in pipelined execution.

In general, there are three major difficulties that cause the instruction pipeline to deviate from its normal operation.

1. *Resource conflicts* caused by access to memory by two segments at the same time. Most of these conflicts can be resolved by using separate instruction and data memories.
2. *Data dependency* conflicts arise when an instruction depends on the result of a previous instruction, but this result is not yet available.
3. *Branch difficulties* arise from branch and other instructions that change the value of PC.

1. Resource Conflict (Same resource needed at same time)

Two instructions want to use the **same hardware (like memory)** at the same time.

Example:

- Instruction 1 is **fetching instruction (FI)** from memory
- Instruction 2 is **fetching data (FO)** from memory
- Both need memory at same time → **conflict happens**

2. Data Dependency (Depends on previous result)

Next instruction needs the result of previous instruction, but result is **not ready yet**.

Example:

- I1: $A = B + C$
I2: $D = A + E$
- Instruction 2 needs value of **A**
- But Instruction 1 has not finished → so **Instruction 2 must wait**

3. Branch Problem (Change in flow)

Branch instruction (like IF, GOTO) changes the next instruction.

Example:

- IF ($A > B$) go to Line 10
- CPU doesn't know next instruction until condition is checked
- So pipeline may fetch **wrong instructions** → need to discard them

RISC Pipeline & Three-Segment Instruction Pipeline

In **RISC (Reduced Instruction Set Computer)**, the instruction set is **very simple and small**.

- All instructions are of **fixed length** (same size)
- Each instruction is divided into **small steps (sub-operations)**
- Each step can be completed in **one clock cycle**

Because of this simplicity:

- It is **easy to design pipeline**
 - Instructions move smoothly from one stage to another
 - **Less delay and better speed**
-
- **RISC uses simple and fixed-size instructions, so pipelining becomes easy, fast, and efficient.**

RISC Pipeline & Three-Segment Instruction Pipeline

- The instruction cycle can be divided into three sub operations and implemented in three segments:
 - **I: Instruction fetch**
 - **A: ALU operation**
 - **E: Execute instruction**
- The I segment **fetches the instruction** from program memory.
- The instruction **is decoded and an ALU operation is performed in the A segment.**
- The E segment directs the output of the ALU to one of three destinations,

- Below figure shows the same program with a no-op instruction inserted after the load to R2 instruction. The data is loaded into R2 in clock cycle 4.
- The add instruction uses the value of R2 in step 5. Thus the no-op instruction is used to advance one clock cycle in order to compensate for the data conflict in the pipeline

Delayed Load

Pipeline timing with data conflict

Clock cycles:	1	2	3	4	5	6
Load R1	I	A	E			
Load R2		I	A	E		
Add R1+R2			I	A	E	
Store R3				I	A	E

Clock cycles:	1	2	3	4	5	6	7
Load R1	I	A	E				
Load R2		I	A	E			
No-operation			I	A	E		
Add R1+R2				I	A	E	
Store R3					I	A	E

Pipeline timing with delayed load

Vector Processing

- Vector processing means performing the **same operation on multiple data at the same time**
- Instead of working on **one value (scalar)**, it works on a **group of values (vector)**
- Used in applications where **large amount of data** is processed
- It is **faster than scalar processing**
- It **reduces execution time**

Applications of Vector Processing (Simple)

- **Weather Forecasting** – Predicting weather for many locations
- **Petroleum Exploration** – Finding oil and gas
- **Seismic Data Analysis** – Earthquake study
- **Medical Diagnosis** – Scanning and analysis
- **Aerodynamics & Space Simulation** – Aircraft and space research
- **Artificial Intelligence** – Smart systems and decision making
- **Human Genome Mapping** – DNA analysis
- **Image Processing** – Editing and analysing images

- A vector V of length n is represented as row vector by
- $$V = [V_1 \quad V_2 \quad V_3 \quad \cdots \quad V_n]$$
- Let us consider the program in assembly language that two vectors A and B of length 100 and put the result in vector C .

- The element V_i of vector V is written as $V(I)$ and the index I refers to a memory address or register where the number is stored.

Using Vector Processing

- It allows operations to be specified with a single vector instruction of the form:

$$C(1:100) = A(1:100) + B(1:100)$$

```

Initialize I = 0
20 Read A(I)
   Read B(I)
   Store C(I) = A(I) + B(I)
   Increment I = I + 1
   If I ≤ 100 go to 20
   Continue

```

Figure Instruction format for vector processor.

Operation code	Base address source 1	Base address source 2	Base address destination	Vector length
----------------	-----------------------	-----------------------	--------------------------	---------------

Matrix Multiplication



- Let us consider the multiplication of two 3×3 matrix A and B.

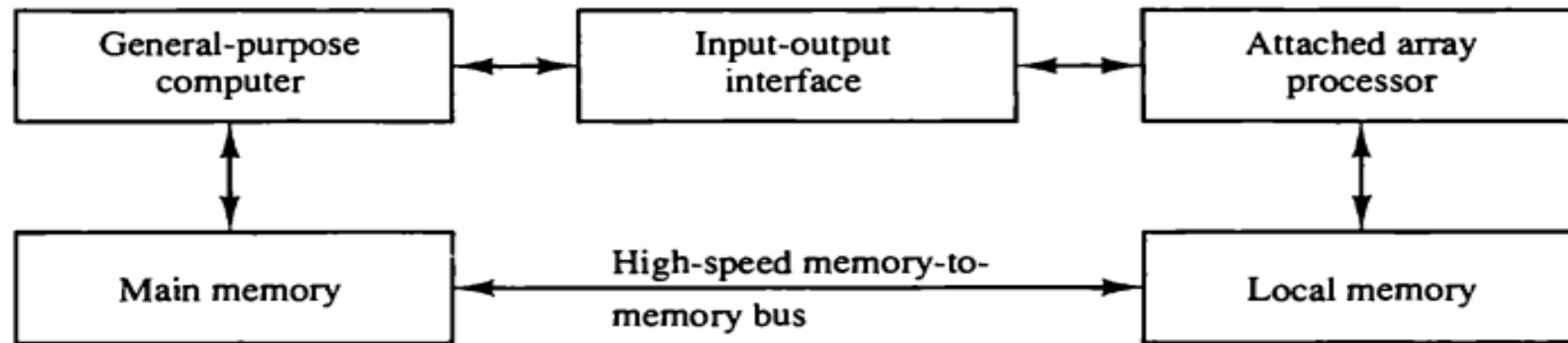
$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

$$C_{11} = (a_{11} \times b_{11}) + (a_{12} \times b_{21}) + (a_{13} \times b_{31})$$

$$C_{12} = (a_{11} \times b_{12}) + (a_{12} \times b_{22}) + (a_{13} \times b_{32})$$

Array Processor

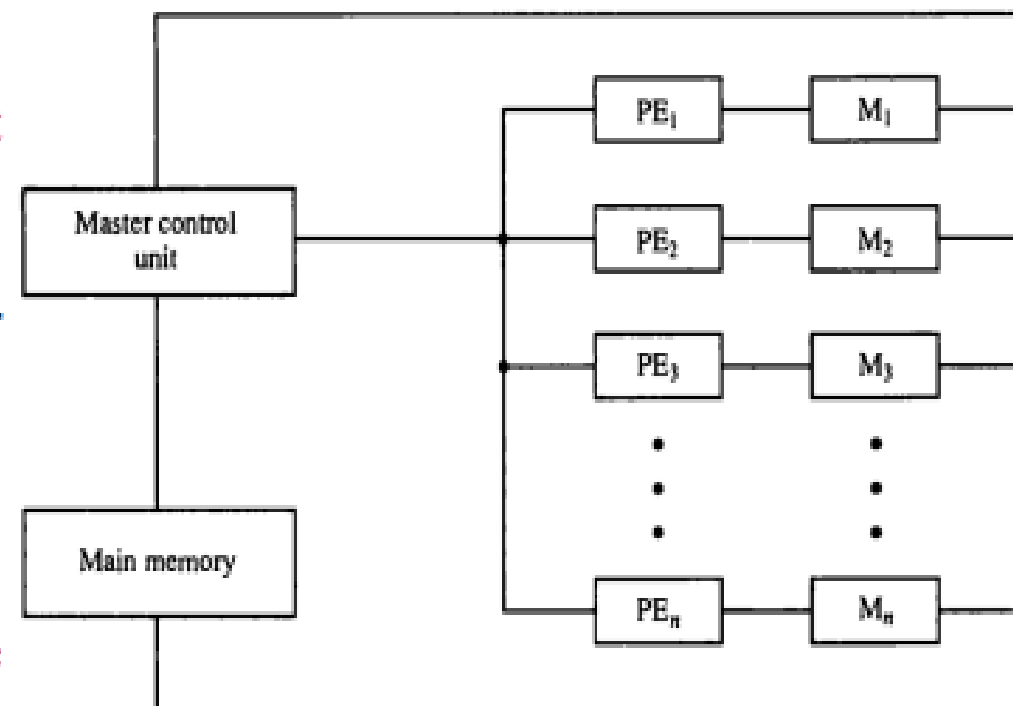
- ♦ An array processor is a processor that performs computations on large arrays of data.
- ♦ Types of array processors:
 - ♦ **1) An attached array processor** is an auxiliary processor attached to a general-purpose computer.
 - ♦ It is intended to improve the performance of the host computer in specific numerical computation tasks.
 - ♦ **2) An SIMD array processor** is a processor that has a single-instruction multiple-data organization.
 - ♦ It manipulates vector instructions by means of multiple functional units responding to a common instruction.



- ♦ The host computer is a general-purpose computer and the attached processor is a back-end machine driven by the host computer.
- ♦ The array processor is connected through an input-output controller to the computer and the computer treats it like an external interface.
- ♦ The data for the attached processor are transferred from main memory to a local memory through a high-speed bus.

- ◆ **SIMD Array Processor:**

- ◆ It is a computer with multiple processing units operating in parallel.
- ◆ The processing units are synchronized to perform the same operation under the control of a common control unit, thus providing a single instruction stream, multiple data stream (SIMD) organization.
- ◆ It contains a set of identical processing elements (PEs), each having a local memory M.
- ◆ Each processor element includes an ALU, a floating-point arithmetic unit, and working registers.
- ◆ The master control unit controls the operations in the processor elements.
- ◆ The main memory is used for storage of the program.
- ◆ The function of the master control unit is to decode the instructions and determine how the instruction is to be executed.



QUESTIONS ASKED IN GTU EXAM

Q.1 Explain Flynn's classification for computers in brief.

Q.2 Explain pipeline conflicts in brief.

Q.3 Write a note on SIMD array processor.

Q.4 Summarize major hazards in pipelined execution.

Q.5 Demonstrate four-segment instruction pipeline in detail

Q.6 Elaborate flynn's classification scheme with proper diagram.

Q.7 Explain arithmetic pipeline.

Q.8 Describe pipeline conflicts.

Q.9 Define pipelining. For arithmetic operation ($A_i * B_i + C_i$) with a stream of seven numbers ($i=1$ to 7). Specify a pipeline configuration to carry out this task.

Q.10 List and explain major instruction pipeline conflicts.

Q.11 Which are different pipeline conflicts. Describe.

Q.12 Write a note on arithmetic pipeline